

Week9_예습과제_이재린

태그

Ch10. 비전 트랜스포머

prep

ViT : 이미지 인식을 위한 딥러닝 모델 >> CNN 모델의 합성곱 계층 방법을 사용하지 않고 트랜스포머 모델에서 사용되는 셀프 어텐션을 적용 like 자연어 처리 방식

but 이미지 패치 원→오, 위→아래 방향으로 입력되어 2차원 구조의 이미지 특성을 온전하게 반영하지 X

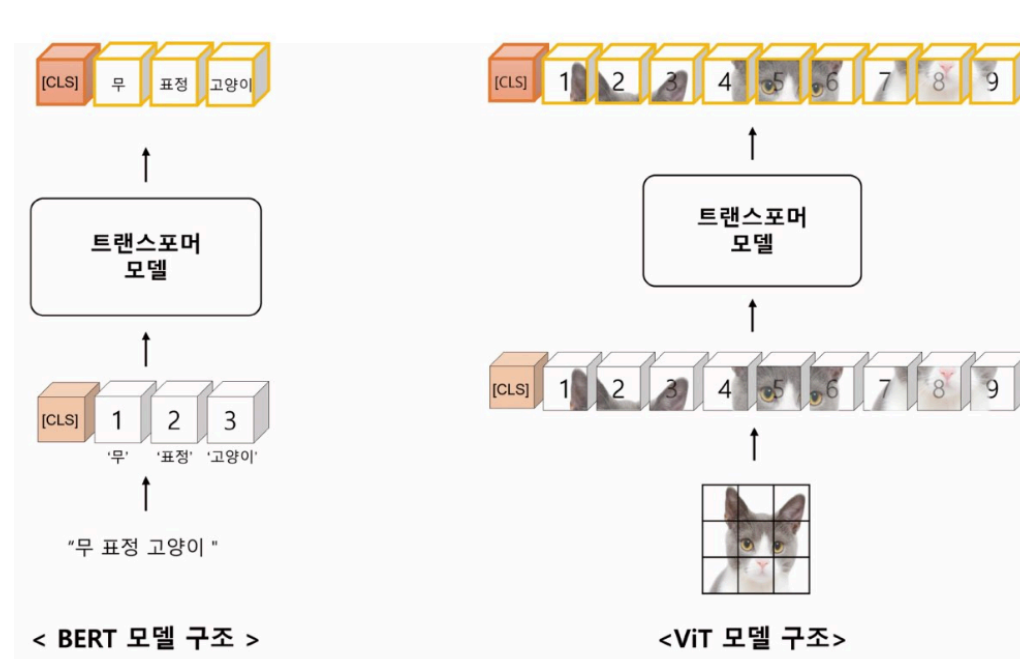
대안

1. swin transformer : Local Window를 활용하여 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지의 특징을 학습 >> 어텐션 함수에는 상대적 위치 편향을 반영하여 자체에 위치적 정보를 포함
2. CvT(Convolutional Vision Transformer) : 기존 합성곱 연산 과정을 ViT에 적용한 모델, 저수준 특징과 고수준 특징을 계층적으로 반영 가능 >> 어텐션 연산 과정에서 키와 값을 기존 벡터보다 축소해 계산 복잡도 감소 가능

1. ViT

BERT 모델 vs ViT 모델

- BERT 모델 : 문장보다 작은 단위의 토큰들이 순차적으로 입력
- ViT 모델 : 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 입력



합성곱 모델과 ViT 모델 비교

: 합성곱 신경망과 트랜스포머는 이미지 특성을 잘 표현하는 임베딩을 만들고자함>> 과정에서 차이

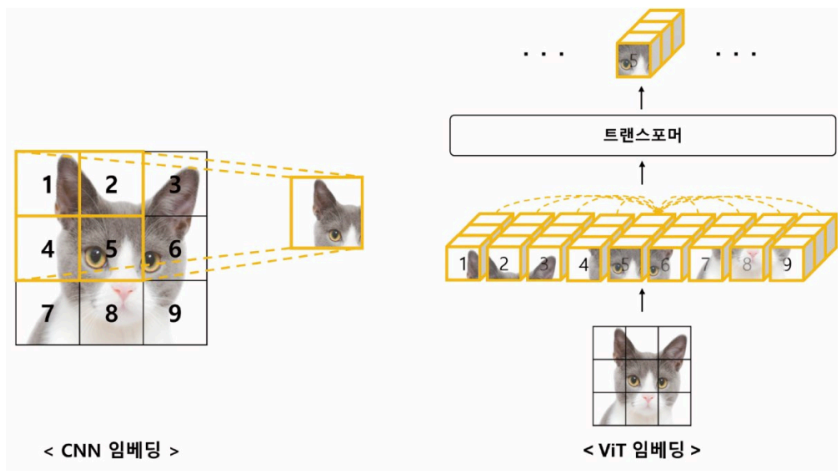


그림 10.3 합성곱 신경망과 ViT 계층의 차이

- 합성곱 임베딩
 - 이미지 패치 중 일부분만 선택하여 학습 >> 전체의 특징을 추출
 - 전체 이미지 정보를 표현하는데 **수많은 계층** 필요
 - 공간적인 위치 정보 고려
- ViT 임베딩
 - 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계를 학습 >> 셀프 어텐션을 통해 서로에게 주는 영향을 고려해 이미지의 전체 특징을 추출 >> 모든 이미지 패치가 학습에 관여
 - 어텐션 거리만 계산 > **한 개의 ViT 레이어**로 전체 이미지 정보 표현 가능
 - 입력 이미지의 크기가 고정 > 이미지 크기 전처리 필요
 - 패치 간의 상대적인 위치 정보 고려 > 이미지 변환에 취약

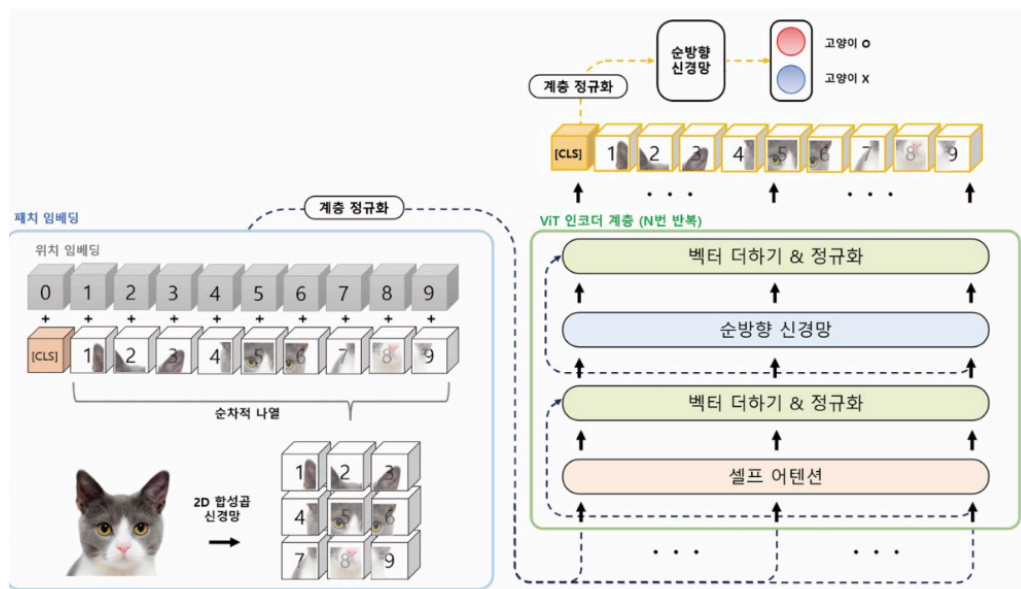
ViT의 귀납적 편향

- 귀납적 편향 : 일반화 성능 향상을 위한 모델의 가정
 - ViT : 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 일반화된 관계를 학습하기 때문엔 귀납적 편향이 거의 없다.
- >> 모델이 가지는 구조와 매개변수들이 데이터에 적합한 가정을 하고 있음을 드러냄 >> 적은 데이터로 일반화 성능이 높아짐

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

ViT 모델

- **패치 임베딩 + 인코더**



- 패치 임베딩 : 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환
- 인코더 : 각 패치와의 관계를 학습

패치 임베딩

- 패치 임베딩 : 이미지 크기를 맞추는 전처리 수행 >> 입력 이미지를 작은 패치로 분할하여 시퀀셜 배열로 만들기
 - 패치 사이즈를 계산하는 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

- 자세한 과정
 - 분할된 배열을 왼→오, 위→아래 순서대로 배열 / 배열 가장 왼쪽에 분류 토큰(전체 이미지를 대표하는 벡터로 특정 문제르 예측하는데 사용) 추가
 - 위치 임베딩 : 인접한 패치 간의 관계 학습 >> 패치의 위치 정보를 임베딩 벡터로 변환하고 기존의 이미지 패치 벡터들과 더한다
 - 계층 정규화 >> 패치 임베딩

인코더 계층

- 벡터로 변환된 패치를 인코더 계층에 입력으로 전달해 다양한 쿼리, 키, 값 임베딩의 관계를 학습
- N개의 인코더 레이어를 반복적으로 적용 > 마지막 레이어에 분류 토큰이라 불리는 패치의 특징 벡터 추출
- 분류 토큰 벡터를 순방향 신경망(완전 연결 계층)에 연결

모델 학습

- 매개 변수

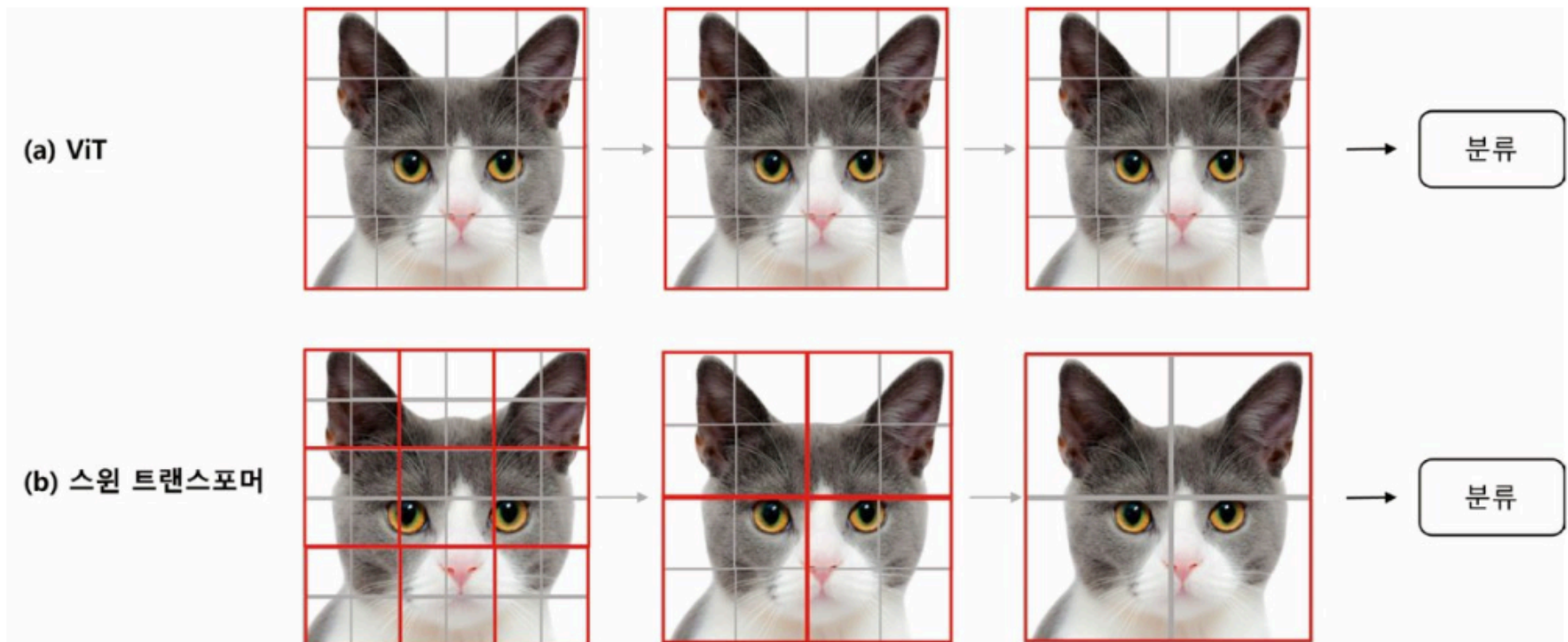
매개변수	의미
output_dir	체크포인트 저장 경로
save_strategy	체크포인트 저장 간격 설정 ▪ no: 저장 안 함 ▪ steps: 스텝마다 ▪ epoch: 에폭마다
save_steps	save_strategy= "steps"일 때 스텝 간격 설정
save_total_limit	체크포인트 최대 저장 개수
evaluation_strategy	체크포인트 평가 간격 설정 ▪ no: 저장 안 함 ▪ steps: 스텝마다 ▪ epoch: 에폭마다

매개변수	의미
learning_rate	초기 학습률
per_device_train_batch_size	학습 배치 크기
per_device_eval_batch_size	평가 배치 크기
num_train_epochs	학습 반복 수
weight_decay	가중치 감쇠
load_best_model_at_end	모델 불러오기 시 최상의 모델 선택 여부
metric_for_best_model	최상의 모델 선정 기준이 되는 평가 방식 설정 ▪ accuracy: 정확도 ▪ precision: 정밀도 ▪ f1: F1 점수 ▪ mae: 평균 절대 오차 ▪ mse: 평균 제곱 오차 ▪ rmse: 평균 제곱근 편차
logging_dir	로그 저장 폴더
logging_steps	로그 출력 간격
seed	학습 시작 시 설정되는 무작위 시드
fp16 ⁵	Mixed precision 사용 여부

2. Swin Transformer

- 기존의 트랜스포머를 기반으로 하는 모델 >> 고정된 패치 크기로 세밀한 예측을 필요로 하는 의미론적 분할과 같은 작업에 적합하지 X
- swin transformer : 계층적 특징 맵을 구성해 Linear 계산 복잡도를 갖는다.
 - shifted window : 입력 이미지를 일정한 크기의 패치로 분할하고 쌓아 윈도우를 구성 >> 여기에만 셀프 어텐션 계산
 - 패치를 겹쳐 효율성 더 큼

ViT와 스윈 트랜스포머 차이

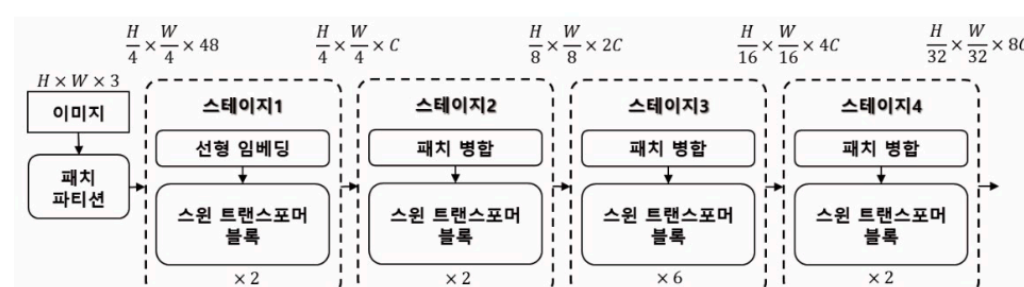


- ViT 모델
 - 고정된 패치 크기로 다양한 이미지 크기와 종횡비를 가진 데이터에 적용하기 어려움
 - 패치 단위의 입력 >> 공간 정보가 완전히 보존되지 않음
 - 패치 수가 증가하면 계산량도 증가
- 스윈 트랜스포머(회색 격자 > 패치, 빨간색 격자 > 로컬 윈도우)
 - 로컬 윈도우를 활용해 물체의 크기나 해상도를 계층적으로 학습 >> 각 패치의 세부 정보에 집중 가능
 - 로컬 윈도우를 이동시키는 시프트 윈도우 기술 : 입력 이미지를 일정 패치로 분할 + 특정 맵을 이동시켜 다음 계층에 사용 >> 전역 특징을 확인 가능, 고해상도 이미지를 효율적으로 처리

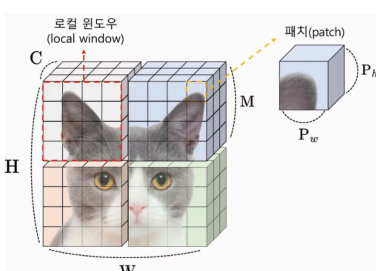
	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

- 모델링 과정



1) 이미지를 패치 파티션으로 구분



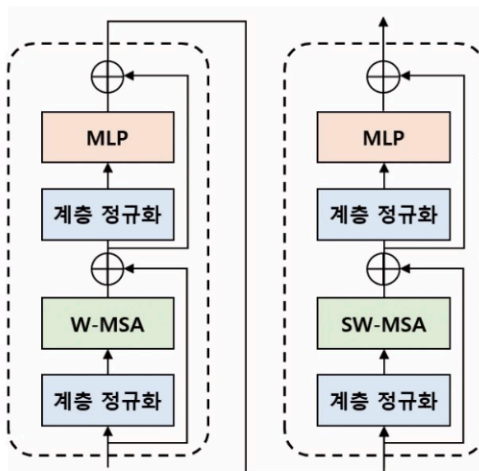
2) 선형 임베딩을 각 패치에 수행 > 각 패치는 고정된 차원의 벡터로 변환

3) 분할된 패치를 기반으로 스윈 트랜스포머 블록을 구성(어텐션 계층, 계층 정규화, 다층 퍼셉트론 등을 포함) > 패치 간 상호작용을 수행

4) 패치를 병합해 전체 이미지에 대한 분류 수행(순차적으로 합치면서 이미지의 전체적인 정보를 추출하는 방식 사용)

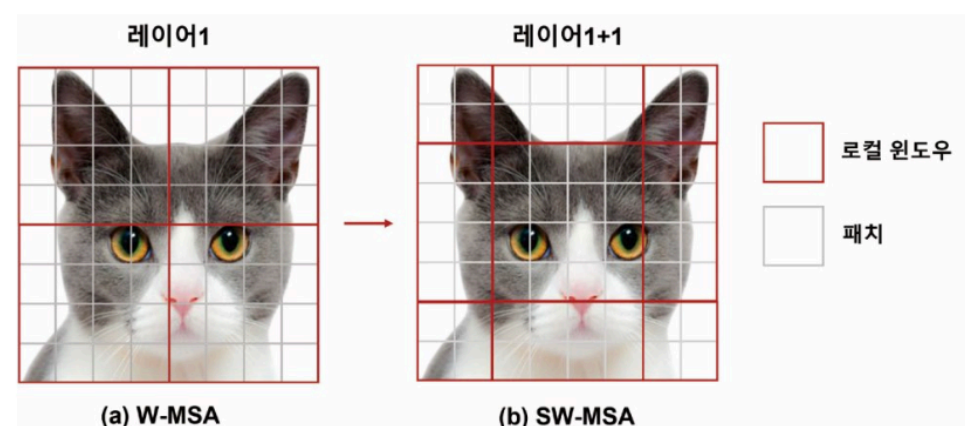
- 패치 병합 : 모델의 매개변수를 줄이기 위해 이미지 텐서 $[C, H, W] \rightarrow [2C, H/2, W/2]$

- 스윈 트랜스포머 블록



*MSA : 기존 트랜스포머 모델에서 사용

- W-MSA : 로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션 >> 입력 특징 맵을 중첩되지 않게 나누고 나누어진 윈도우 영역에서 독립적으로 셀프 어텐션 수행
 - 연산량 감소 but 윈도우 내부의 이미지 패치 영역에서만 셀프 어텐션 수행
- SW-MSA : W-MSA를 보완, 윈도우 사이에 연결성 구축해 윈도우 간의 관계성 정보 추출



3. CvT

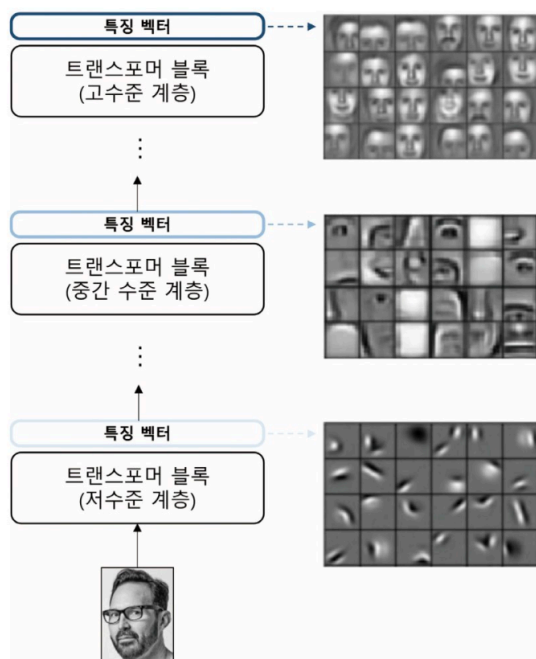
CvT : 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델

>> ViT의 셀프 어텐션의 이미지 정보를 바탕으로 각 패치 간의 관계를 학습하지만, 모든 패치에 대해 동일한 크기와 간격을 사용하여 물체의 크기나 해상도를 계층적으로 학습하기가 어려운 점을 보완

합성곱 계층의 지역 특징을 추출+ ViT의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지를 처리

1. CvT 모델의 계층적 특징

: 이미지를 처리하기 전 합성곱 연산을 통해 특징 맵 추출 > 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용 & 겹쳐서 학습

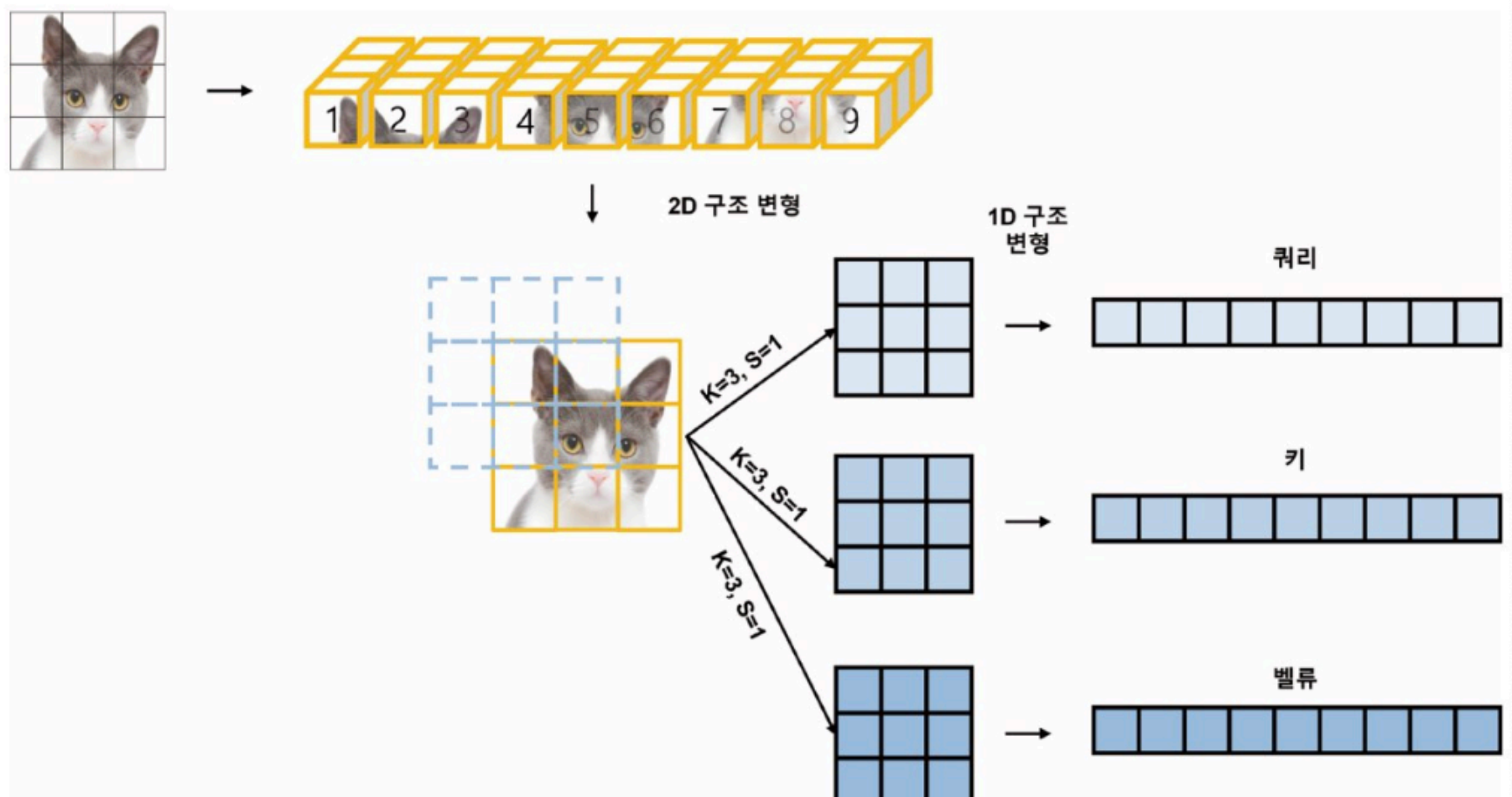


Ex)

- 저수준 계층 : 선 또는 점에 대한 특징 표현
- 중간 수준 : 눈이나 귀와 같은 특징 표현
- 고수준 : 얼굴 구조에 대한 특징 표현

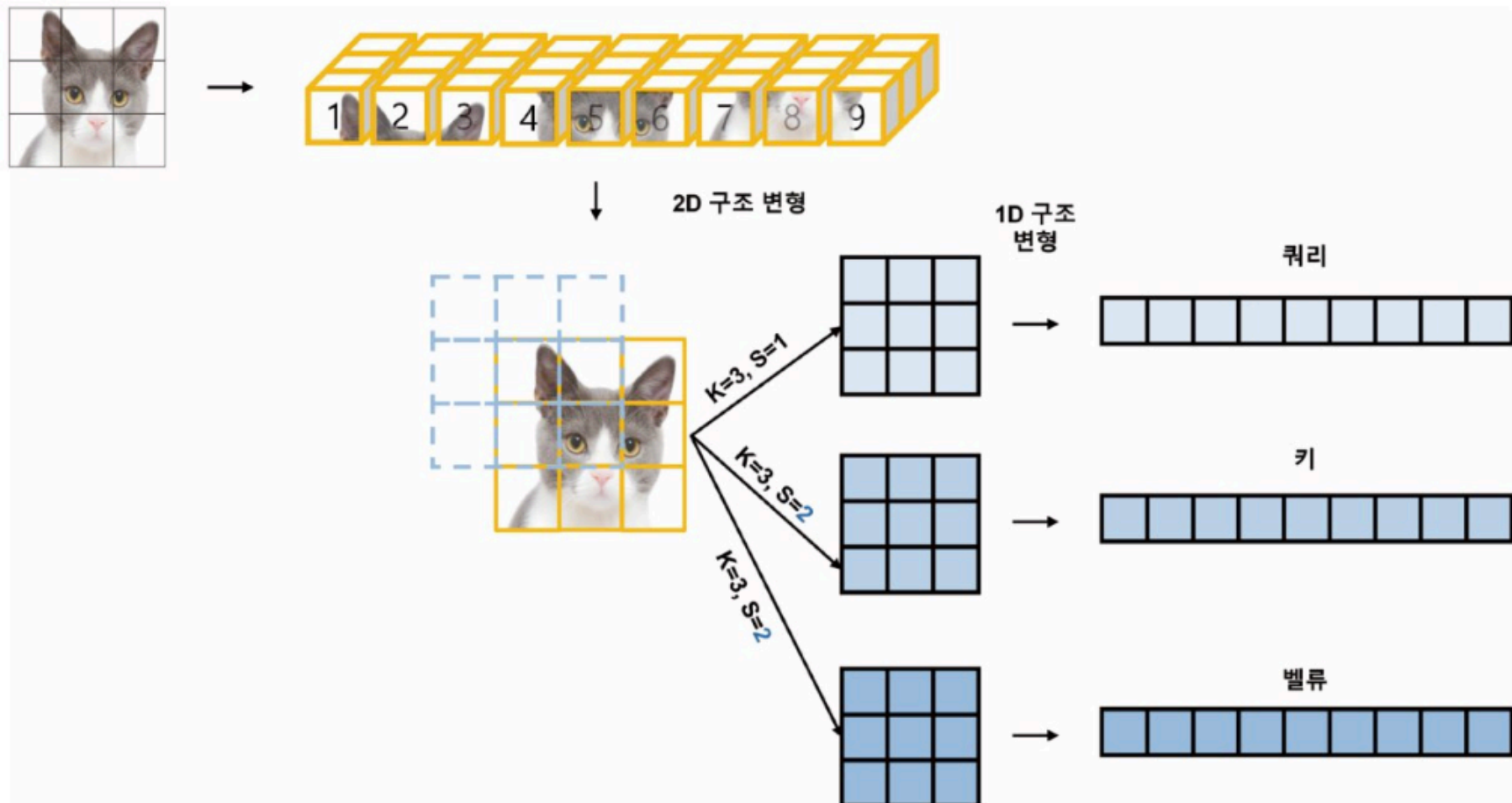
모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

2. 합성곱 토큰 임베딩



- 합성곱 토큰 임베딩이란? 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩 >> ViT에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용
- 기존에 비해 CvT모델은 2D 구조를 보존 & 셀프 어텐션을 계산 >> 이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, value 임베딩으로 변환 >> 이걸로 셀프 어텐션 계산하고 출력 생성

3. 어텐션에 대한 합성곱 임베딩



↳ 쿼리 : 3×3 & stride=1 인 커널로 4개의 이미지 패치를 생성

↳ 키와 value : 3×3 & stride=2 인 커널로 9개의 이미지 패치를 생성

: 2D로 재구성된 토큰맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산

: 모델의 매개변수의 수를 줄여 쿼리, 키, 값의 차원을 축소

